

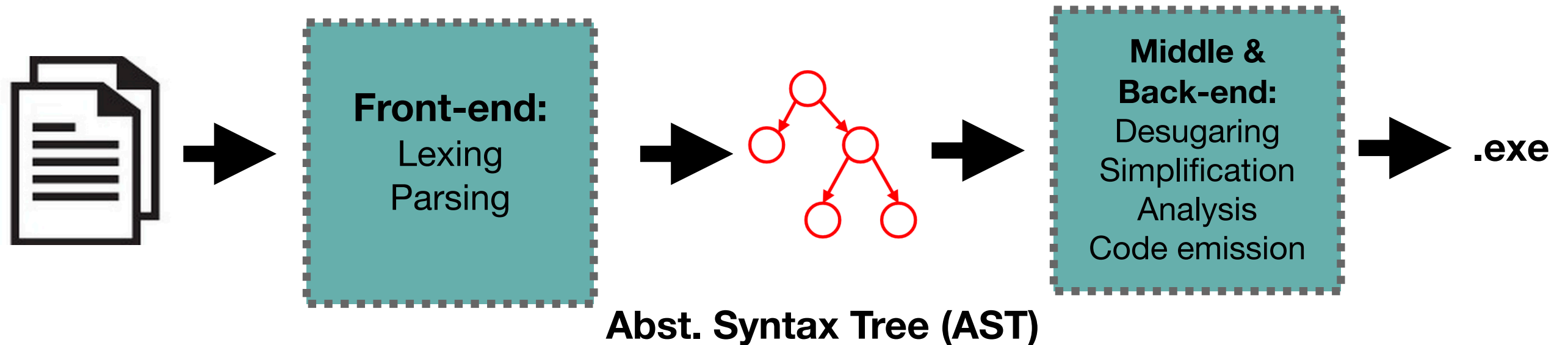
Introduction to Languages, Syntax, Lexing and Parsing

CptS 355 — Fall 2026

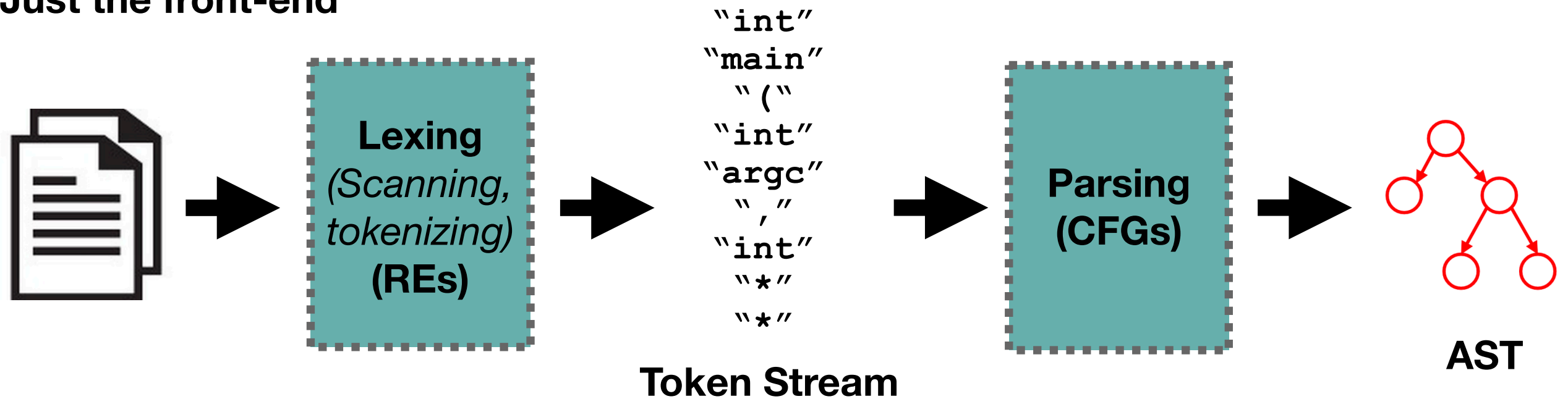
Thomas Gilray

Typical (big-picture) organization of an interpreter:

Interpreter/Compiler organization



Just the front-end



Discrete Math and Notation

review

Propositional Logic

A ***proposition*** is a logical statement that is true or false

Propositional Logic

A ***proposition*** is a logical statement that is true or false

The ***disjunction*** of two propositions is true if and only if either or both is true

$$P \vee Q$$

Propositional Logic

A ***proposition*** is a logical statement that is true or false

The ***disjunction*** of two propositions is true if and only if either or both is true

$$P \vee Q$$

The ***conjunction*** of two propositions is true if and only if both are true

$$P \wedge Q$$

Propositional Logic

A **proposition** is a logical statement that is true or false

The **disjunction** of two propositions is true if and only if either or both is true

$$P \vee Q$$

The **conjunction** of two propositions is true if and only if both are true

$$P \wedge Q$$

The **negation** of a proposition P is true if and only if P is not

$$\neg P$$

Propositional Logic

A **proposition** is a logical statement that is true or false

The **disjunction** of two propositions is true if and only if either or both is true

$$P \vee Q$$

The **conjunction** of two propositions is true if and only if both are true

$$P \wedge Q$$

The **negation** of a proposition P is true if and only if P is not

$$\neg P$$

The **predicate** P is a function that yields true/false (a predicate is an abstraction of a proposition)

$$P(x)$$

Propositional Logic

An ***implication*** says that one proposition (Q) must hold true if another (P) does

$$P \implies Q$$

Propositional Logic

An **implication** says that one proposition (Q) must hold true if another (P) does

$$P \implies Q$$

Implications can also be written in terms of negation and disjunction

$$\neg P \vee Q$$

Propositional Logic

An **implication** says that one proposition (Q) must hold true if another (P) does

$$P \implies Q$$

Implications can also be written in terms of negation and disjunction

$$\neg P \vee Q$$

$P \implies Q$		
	Q	
P	false	true
false	true	true
true	false	true

$\neg P \vee Q$		
	Q	
P	false	true
false	true	true
true	false	true

Propositional Logic

An **implication** says that one proposition (Q) must hold true if another (P) does

$$P \implies Q$$

Implications can also be written in terms of negation and disjunction

$$\neg P \vee Q$$

The **converse** (reverse) of an implication is **not** equivalent to the original

$$Q \implies P$$

The **contrapositive** of an implication **is** equivalent to the original

$$\neg Q \implies \neg P$$

Try an Example:

Draw the truth table for $\neg A \vee \neg B$

Try an Example:

Draw the truth table for $\neg A \vee \neg B$

		A	
		false	true
B	false	true	true
	true	true	false

Try an Example:

Draw the truth table for $\neg A \wedge B \implies C$

Try an Example:

Draw the truth table for $\neg A \wedge B \implies C$

		C	
		false	true
A,B	false, false		
	false, true		
	true, false		
	true, true		

Try an Example:

Draw the truth table for $\neg A \wedge B \implies C$

		C	
		false	true
A,B	false, false	true	true
	false, true	false	true
	true, false	true	true
	true, true	true	true

Sets

A set is a collection of zero or more values, without duplicates. A set itself is also a value that may be found in another (higher-order) set.

$\{1,2,3\}$  **A set containing three integers**

Sets

A set is a collection of zero or more values, without duplicates. A set itself is also a value that may be found in another (higher-order) set.

$\{1,2,3\}$  **A set containing three integers**

$\emptyset$ or $\{\}$  **An empty set is written two ways**

Sets

A set is a collection of zero or more values, without duplicates. A set itself is also a value that may be found in another (higher-order) set.

$\{1,2,3\}$

A set containing three integers

$\emptyset$ or $\{\}$

An empty set is written two ways

$1 \in \{1,2\}$

A proposition stating 1 is in the set $\{1,2\}$

Sets

A set is a collection of zero or more values, without duplicates. A set itself is also a value that may be found in another (higher-order) set.

$\{1,2,3\}$

A set containing three integers

$\emptyset$ or $\{\}$

An empty set is written two ways

$1 \in \{1,2\}$

A proposition stating 1 is in the set $\{1,2\}$

The left-hand expression evaluated to a value in the set for each assignment of free variables.

The right-hand expression is a proposition that guards these values.

Set-builder notation

$\{2x \mid x \in \mathbb{N} \wedge x > 10\}$

Sets

$\{22, 24, 26, 28, 30, 32, \dots\}$

The left-hand expression evaluated to a value in the set for each assignment of free variables.

The right-hand expression is a proposition that guards these values.

Set-builder notation

$\{2x \mid x \in \mathbb{N} \wedge x > 10\}$

Sets and Logic

Sets and logical predicates are closely related. **A predicate can encode a set** (by returning true only for elements of the set), and, likewise, **a set can encode a predicate** by containing those elements accepted by the predicate.

$$P(x) = (x \in \mathbb{N} \wedge x > 10)$$

The right-hand expression
is a proposition that guards
these values.


$$\{2x \mid x \in \mathbb{N} \wedge x > 10\}$$

Propositional Logic and Sets

DeMorgan's laws show how conjunction and disjunction are related, via negation.

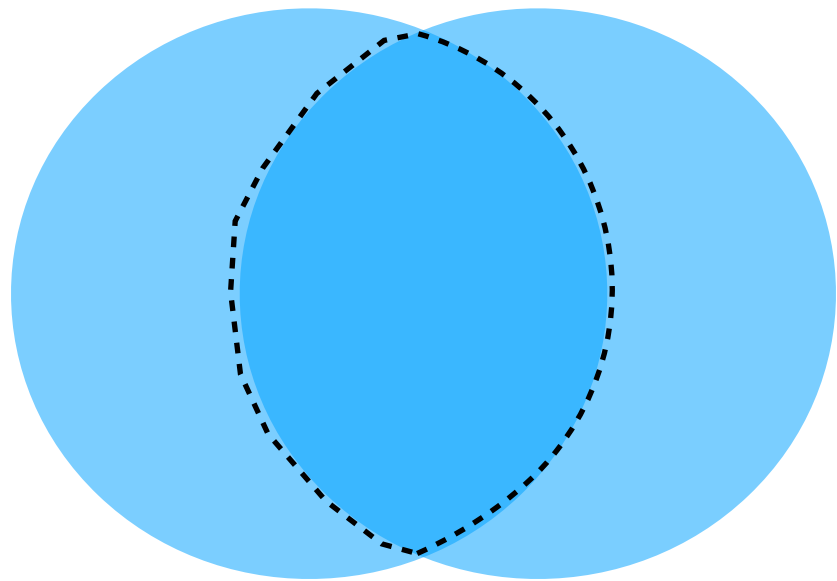
$$a \vee b \iff \neg(\neg a \wedge \neg b)$$

$$a \wedge b \iff \neg(\neg a \vee \neg b)$$

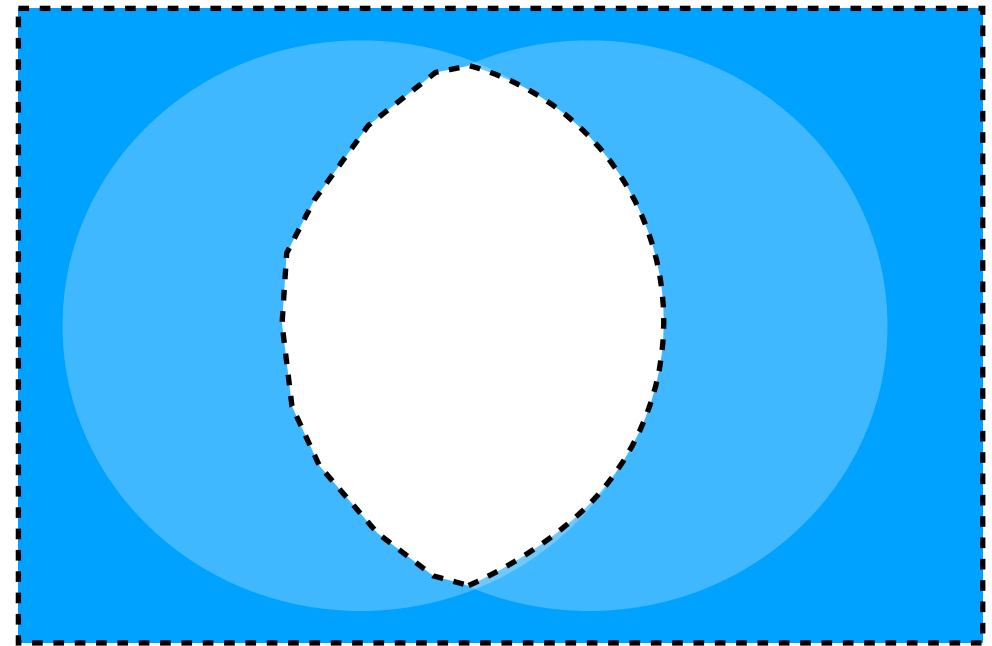
Naturally, this applies to sets as well.

$$A \cap B \iff \overline{(\bar{A} \cup \bar{B})}$$

Propositional Logic and Sets



$$A \cap B$$



$$\overline{A} \cup \overline{B}$$

$$\overline{(\overline{A} \cup \overline{B})}$$

Propositional Logic and Sets

Sets can (nonstrictly or strictly) **include** one another.

$$S_0 \subseteq S_1$$

$$S_0 \subset S_1$$

Every element in S_0 is in S_1

...and some element in S_1 is not in S_0

Note that a predicate defining one set that includes another,
is logically implied by the other's predicate

$$\{x \mid P(x)\} \subseteq \{x \mid Q(x)\}$$

if and only if

$$P(x) \implies Q(x)$$

Try an Example:

Write a set builder definition of: $\{-1, 0, 3, 8, 15, 24, 35, 48, 63, \dots\}$

$$\{n^2 - 1 \mid n \in \mathbb{N}\}$$

First-order Logic and Sets

Quantifiers lift a property of values to a property of sets of those values

Logic systems that include quantification are called ***predicate calculus*** or ***first-order logic***

First-order Logic and Sets

Quantifiers lift a property of values to a property of sets of those values

Logic systems that include quantification are called ***predicate calculus*** or ***first-order logic***

The ***universal quantifier*** is written as an upside-down A and states a proposition holds “for all” elements of a set

$$\forall x \in D . P(x)$$

The ***existential quantifier*** is written as a backward E and states “there exists” a set element for which a proposition holds

$$\exists x \in D . P(x)$$

Sets

N

Natural numbers
 $\{0, 1, 2, 3, 4, \dots\}$

Z

Integers
 $\{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$

P

Positive integers
 $\{1, 2, 3, 4, \dots\}$

C

Complex numbers
e.g., $2i$, $-5 + 2.5i$

R

Real numbers
 $\{1, -0.1, -0.01, 3.1415, \dots\}$

Q

Rational numbers
e.g., $1/2$, $1/3$

Set operations

The ***cardinality*** of a set is its size, or number of elements.

$$|\{0,2,4,6\}| = 4$$

Set operations

The **cardinality** of a set is its size, or number of elements.

$$|\{0,2,4,6\}| = 4$$

But what do we say about the **cardinality** of infinite sets, such as

$$|\mathbb{N}| = ? \qquad |\mathbb{R}| = ?$$

More on this later...

Set operations

The ***union of sets*** yields a set with all elements from any of the operands

$$\{1,2,3\} \cup \{3,4\} = \{1,2,3,4\}$$

Set operations

The ***union of sets*** yields a set with all elements from any of the operands

$$\{1,2,3\} \cup \{3,4\} = \{1,2,3,4\}$$

The ***intersection of sets*** yields a set with those elements all operands share

$$\{1,2,3\} \cap \{3,4\} = \{3\}$$

Set operations

The ***union of sets*** yields a set with all elements from any of the operands

$$\{1,2,3\} \cup \{3,4\} = \{1,2,3,4\}$$

The ***intersection of sets*** yields a set with those elements all operands share

$$\{1,2,3\} \cap \{3,4\} = \{3\}$$

Set difference yields a set with all elements in the first that are not in the second

$$\{1,2,3\} - \{3,4\} = \{1,2\}$$

Set operations

The ***union of sets*** yields a set with all elements from any of the operands

$$\{1,2,3\} \cup \{3,4\} = \{1,2,3,4\}$$

The ***intersection of sets*** yields a set with those elements all operands share

$$\{1,2,3\} \cap \{3,4\} = \{3\}$$

Set difference yields a set with all elements in the first that are not in the second

$$\{1,2,3\} - \{3,4\} = \{1,2\}$$

The ***Cartesian product*** of two sets yields a set of all pairs of elements

$$\{1,2,3\} \times \{A,B\} = \{(1,A), (1,B), \\ (2,A), (2,B), \\ (3,A), (3,B)\}$$

Set operations

$$\mathbb{N} \times \mathbb{N} =$$

$$\begin{aligned} &\{ (0,0), (0,1), (0,2), (0,3), \dots \\ &\quad (1,0), (1,1), (1,2), (1,3), \dots \\ &\quad (2,0), (2,1), (2,2), (2,3), \dots \\ &\quad (3,0), (3,1), (3,2), (3,3), \\ &\quad \vdots \end{aligned}$$

$\ddots$

Set operations

N^2

vs

2^N

Set operations

$$\mathbb{N}^2$$

vs

$$2^{\mathbb{N}}$$

$$= \mathbb{N} \times \mathbb{N}$$

$$= \mathcal{P}(\mathbb{N})$$

The “power set” of the natural numbers

Set operations

The **power set** of a set A is a higher-order set that contains *all subsets* of A .

The power set of A is written 2^A or $\mathcal{P}(A)$

For example:

$$\begin{aligned}\mathcal{P}(\{1,2,3\}) = \{ & \emptyset, \\ & \{1\}, \{2\}, \{3\}, \\ & \{1,2\}, \{2,3\}, \{1,3\}, \\ & \{1,2,3\} \end{aligned}$$

Set operations

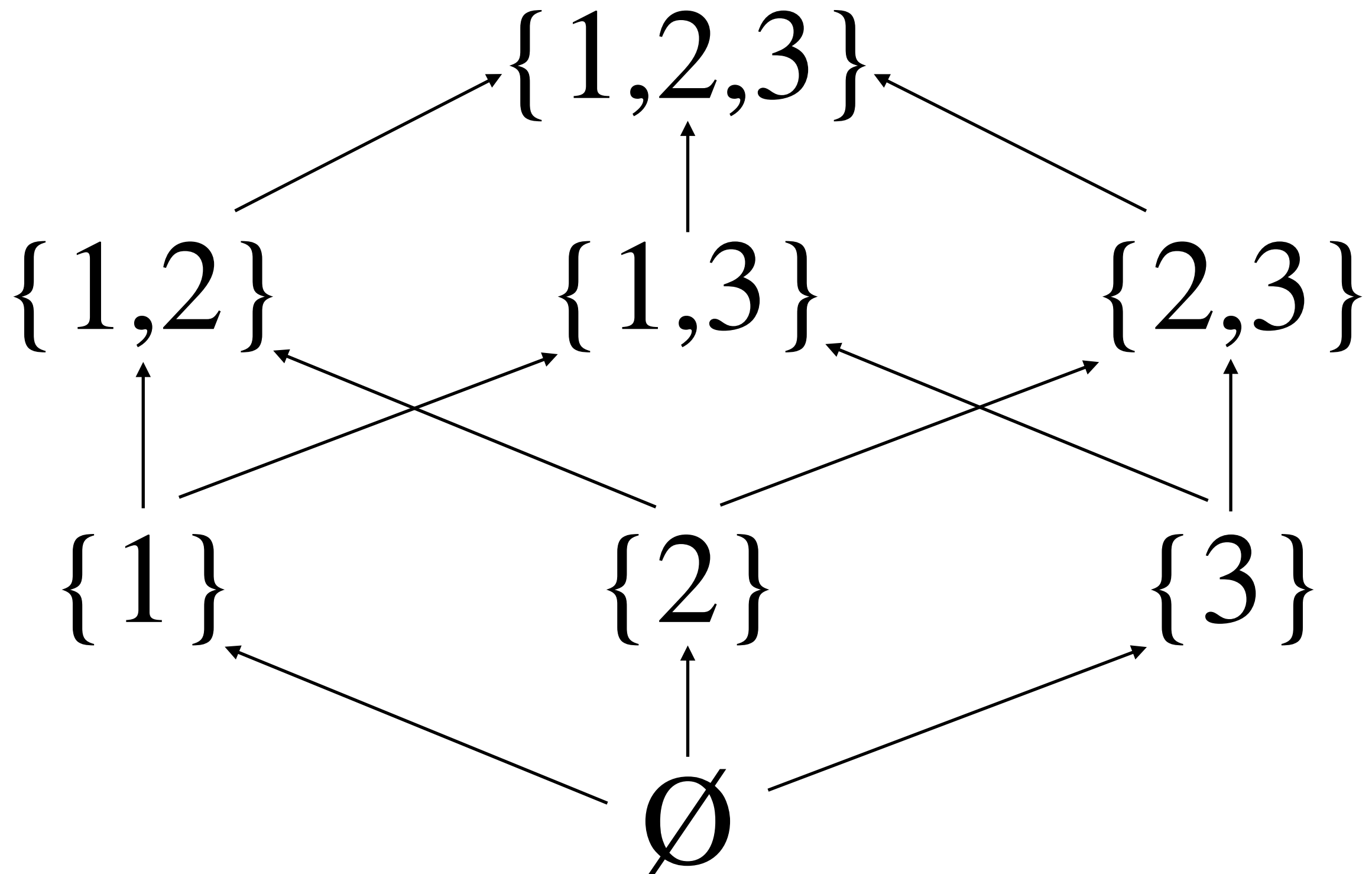
Observe that: $|2^A| = 2^{|A|}$, hence this notation for power set!

For the set $\{1,2,3\}$, with 3 elements, we can think of each subset as a string of 3 bits (one of 000, 001, 010, 011, ..., 111) specifying with a zero or a one which elements are in the subset. In this encoding, 000 represents the empty subset, 010 represents $\{2\}$, and 111 represents $\{1,2,3\}$.

$$\begin{aligned}\mathcal{P}(\{1,2,3\}) = \{ & \emptyset, \\ & \{1\}, \{2\}, \{3\}, \\ & \{1,2\}, \{2,3\}, \{1,3\}, \\ & \{1,2,3\} \end{aligned}$$

$$\mathcal{P}(A) = 2^A =$$

Hasse diagram



$$(\subseteq) \subseteq \mathcal{P}(A) \times \mathcal{P}(A)$$

Relations

A relation is a subset of some Cartesian product.

$$R \subseteq A \times B$$

In this case R is a relation between elements of A and B where
 $a R b$ if and only if $(a, b) \in R$



($\in$) is itself a relation for set-inclusion!

For example, the relation $(<) \subseteq \mathbb{Z} \times \mathbb{Z}$ encoding less-than among integers includes the tuples $(2,4)$ and $(3,4)$ but not $(4,4)$ or $(5,4)$

Properties of Relations

Reflexivity

$$xRx$$

Symmetry

$$xRy \implies yRx$$

Anti-symmetry

$$xRy \wedge yRx \implies x = y$$

Transitivity

$$xRy \wedge yRz \implies xRz$$

F is a function:

$$\forall x, y, z . (x, y) \in F \wedge (x, z) \in F \implies y = z$$

F is a total function:

$$\forall x . \exists y . (x, y) \in F$$

F is injective/1-1:

$$\forall x, y . F(x) = F(y) \implies x = y$$

F is surjective/onto:

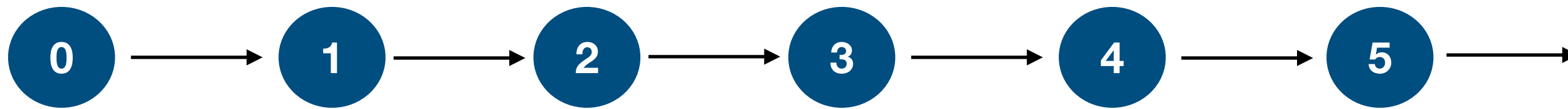
$$\forall y . \exists x . (x, y) \in F$$

$$R = \{(0,1), (1,2), (2,3), (3,4), \dots\}$$

Or, using *set-builder* notation:

$$R = \{(n, n + 1) \mid n \in \mathbb{N}\}$$

$$R = \{(0,1), (1,2), (2,3), (3,4), \dots\}$$

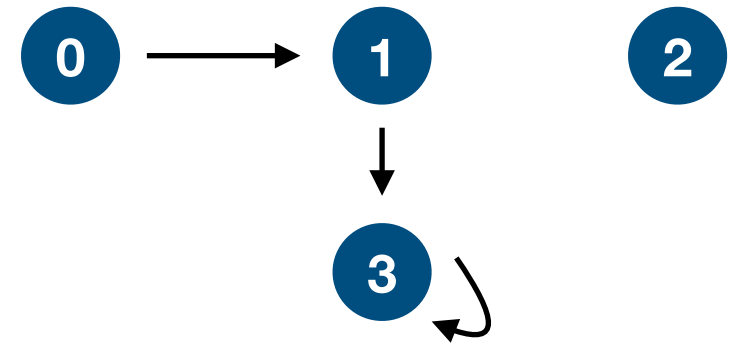


Relations can also be viewed as encoding the edges of a graph

And (connected) graphs can be represented as a relation!

Graphs

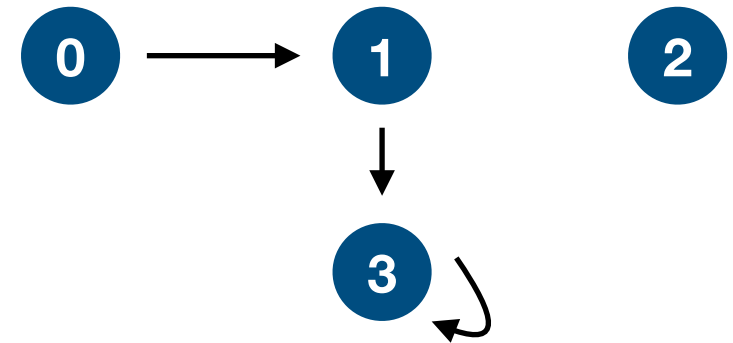
A **graph** is a structure made of **vertices** and **edges**



The graph above can be formalized as $\langle V, E \rangle$,
a set of vertices $V = \{0, 1, 2, 3\}$ and edges $E = \{(0, 1), (1, 3), (3, 3)\}$.

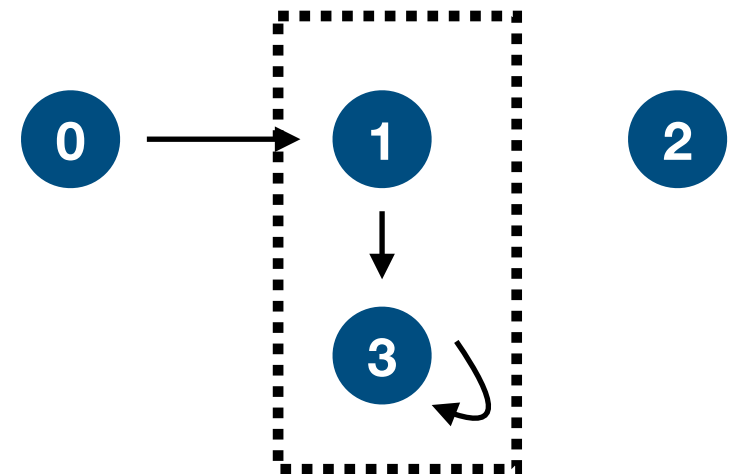
Graphs

A **graph** is a structure made of **vertices** and **edges**



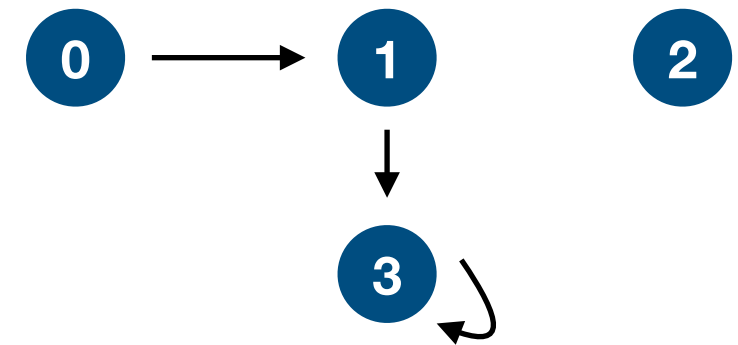
The graph above can be formalized as $\langle V, E \rangle$,
a set of vertices $V = \{0, 1, 2, 3\}$ and edges $E = \{(0, 1), (1, 3), (3, 3)\}$.

A **subgraph** is any subset of a graph's vertices and edges that itself forms a graph

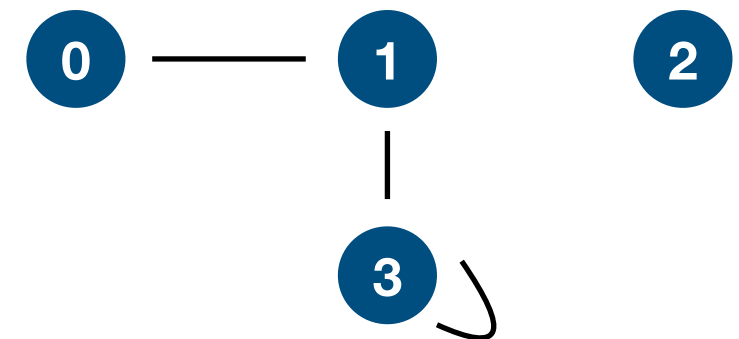


Graphs

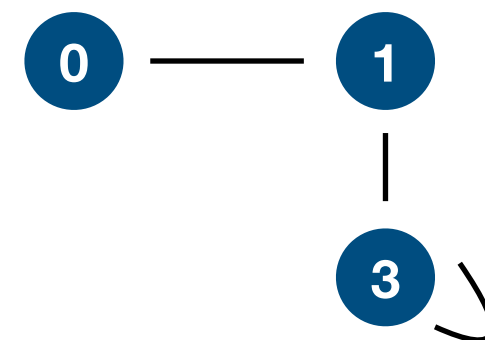
A **path** or **walk** is a sequence of end-to-end edges between two vertices (e.g., 0–1–3–3–3)



An **undirected graph** is a graph where every edge is bidirectional.



A **connected graph** is a graph for which all two vertices are connected by some path



- Partial orders are reflexive, antisymmetric, and transitive.
 - If $\forall x, y . xRy \vee yRx$, then the order is *total*.
 - If it is not antisymmetric, then it's a *preorder*.
 - If it is irreflexive, then it is a *strict* partial order.
- A set S and a partial order ($\sqsubseteq$) form a poset $(S, \sqsubseteq)$ together —a *partially ordered set*.
 - For example: $(\mathbb{R}, \leq)$ or $(\mathcal{P}(\mathbb{N}), \subseteq)$
- An equivalence relation is reflexive, symmetric, and transitive —has *equivalence classes*. What does its graph look like?

Formal Languages

are exact sets of strings

$$L = \{\text{hello, bonjour, konnichiwa, ...}\}$$

$$\Sigma = \{\mathbf{a, b, c, ..., y, z}\}$$

$$\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \dots, \mathbf{y}, \mathbf{z}\}$$

$$\Sigma^* = \{w_1 \dots w_k \mid \forall k, i \in \mathbb{N} . w_i \in \Sigma\}$$

$$\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \dots, \mathbf{y}, \mathbf{z}\}$$

$$\Sigma^* = \{w_1 \dots w_k \mid \forall k, i \in \mathbb{N}. w_i \in \Sigma\}$$

Examples:

$$\mathbf{abcabc} \in \Sigma^*$$

$$\epsilon \in \Sigma^*$$

$$\mathbf{aaaa} \dots \notin \Sigma^*$$

Regular languages

Regular expressions: E

a

The interpretation of regex “a” is the singleton set containing just the string “a”.

$$\mathcal{L}(\mathbf{a}) = \{\mathbf{a}\}$$

All characters in the alphabet are regular expressions.

$$\forall a \in \Sigma . \mathcal{L}(a) = \{a\}$$

There is also an empty/null language, $\emptyset$,
and an empty-string language, ϵ .

$$\mathcal{L}(\emptyset) = \{\}$$

$$\mathcal{L}(\epsilon) = \{\epsilon\}$$

Composite forms of regular expressions can be derived, from other composite forms, and terminally, from null, empty, or single-character REs.

A minimal and sufficient set of derived forms is: *disjunction of REs, composition of REs, and kleene star of REs.*

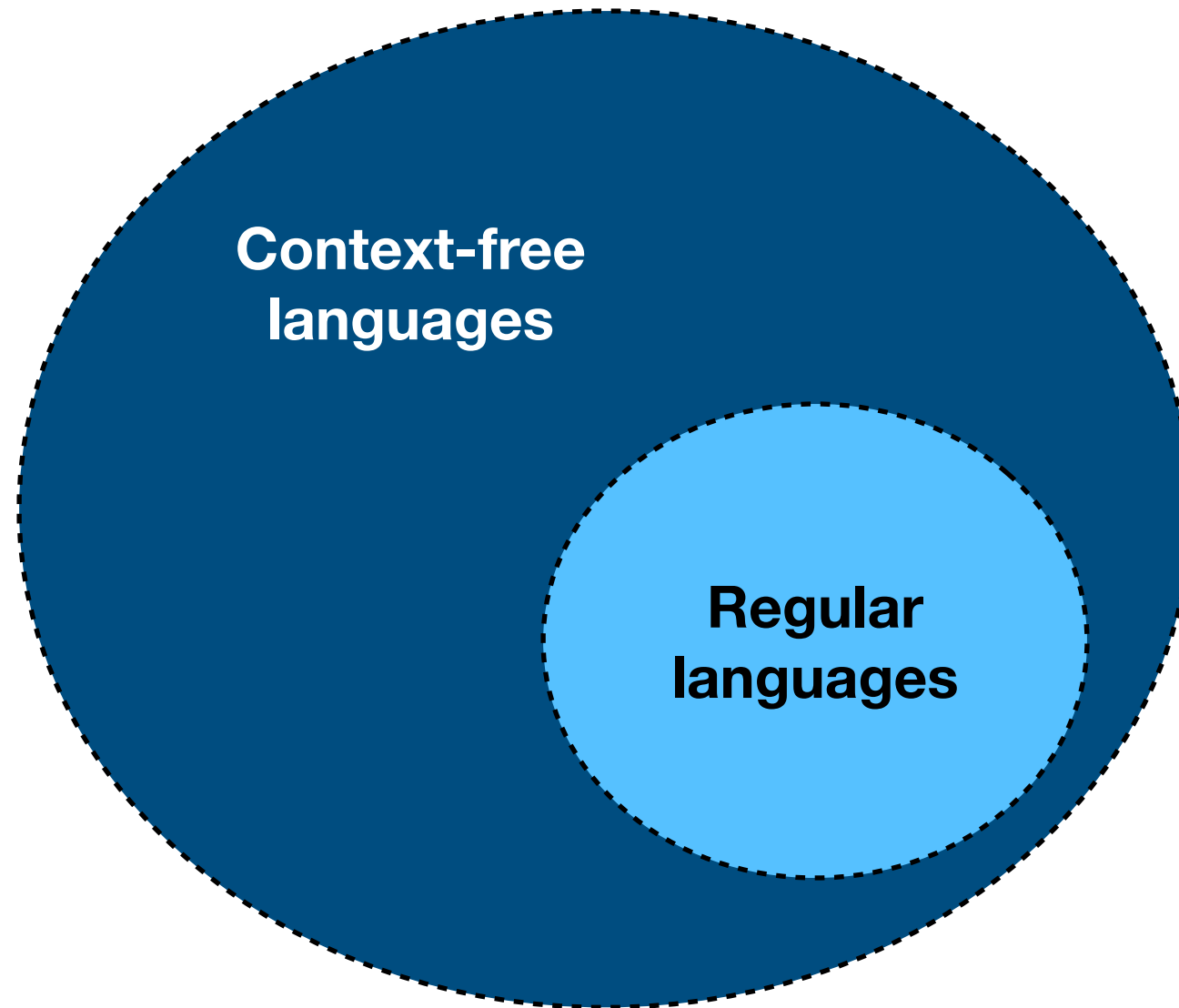
$$\emptyset \in E, \epsilon \in E, a \in \Sigma \implies a \in E$$

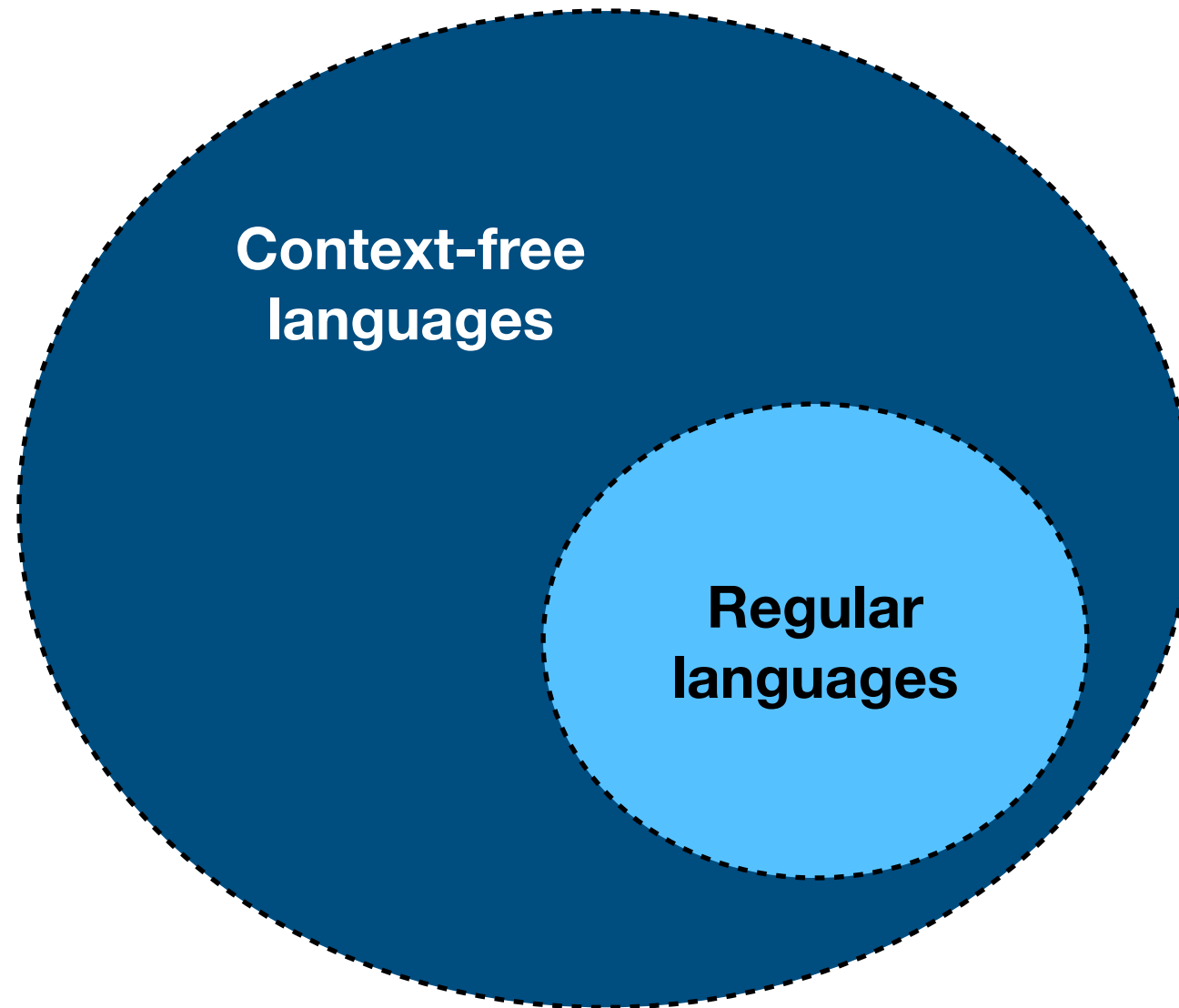
$$e \in E \implies (e) \in E$$

$$e_0 \in E \wedge e_1 \in E \implies e_0 e_1 \in E$$

$$e_0 \in E \wedge e_1 \in E \implies e_0 | e_1 \in E$$

$$e \in E \implies e^* \in E$$





RE (Turing-equivalent computation)

$$L = \{ \mathbf{a}^n \mathbf{b}^n \mid n \in \mathbb{N} \}$$

$$L = \{ \mathbf{a}^n \mathbf{b}^n \mid n \in \mathbb{N} \}$$

$$S \rightarrow \mathbf{a} S \mathbf{b} \mid \epsilon$$


Any string s in the language (denoted by) S may be prepended with $\mathbf{a}$ and appended with $\mathbf{b}$ and the resulting string is also in the language S .

The empty string is in S .

Grammars (shown as BNF, EBNF)

$$e \in E ::= w \mid \emptyset \mid \epsilon \\ \mid (e) \\ \mid e_0 e_1 \\ \mid e_0 + e_1 \\ \mid e^*$$

There is one base case defining regexes and four inductive cases.

Both “|” and “+” are commonly used to signify disjunction in regexes.

$$w \in \Sigma = \{ \dots \}$$

10. Full Grammar specification ¶

This is the full Python grammar, as it is read by the parser generator and used to parse Python source files:

```
# Grammar for Python

# NOTE WELL: You should also follow all the steps listed at
# https://devguide.python.org/grammar/

# Start symbols for the grammar:
#     single_input is a single interactive statement;
#     file_input is a module or sequence of commands read from an input file;
#     eval_input is the input for the eval() functions.
# NB: compound_stmt in single_input is followed by extra NEWLINE!
single_input: NEWLINE | simple_stmt | compound_stmt NEWLINE
file_input: (NEWLINE | stmt)* ENDMARKER
eval_input: testlist NEWLINE* ENDMARKER

decorator: '@' dotted_name [ '(' [arglist] ')' ] NEWLINE
decorators: decorator+
decorated: decorators (classdef | funcdef | async_funcdef)

async_funcdef: 'async' funcdef
funcdef: 'def' NAME parameters ['->' test] ':' suite

parameters: '(' [typedargslist] ')'
typedargslist: (tfpdef ['=' test] (',' tfpdef ['=' test])* [',' [
    '*' [tfpdef] (',' tfpdef ['=' test])* [',' ['**' tfpdef [',']] ] ] ]
```

Interpreting Regexes

Precedence:

kleene star (*), concatenation (ab), then
disjunction (a|b).

Thus, $a|bc|bcd^*$ is the same as
 $(a)|(bc)|(bc(d^*))$

$$\mathcal{L}(a) = \{a\} \quad \mathcal{L}(\epsilon) = \{\epsilon\}$$

$$\mathcal{L}(\emptyset) = \{\}$$

**Juxtaposition is language concatenation, disjunction is language union,
kleene star is interpreted as kleene closure:**

$$\mathcal{L}(e_0 e_1) = \mathcal{L}(e_0) \circ \mathcal{L}(e_1)$$

$$\mathcal{L}(e_0 | e_1) = \mathcal{L}(e_0) \cup \mathcal{L}(e_1)$$

$$\mathcal{L}(e_0^*) = \mathcal{L}(e_0)^*$$

Language concatenation:

$$L \circ L' = \{s_0s_1 \mid s_0 \in L \wedge s_1 \in L'\}$$

Kleene-closure of a Language:

$$L^* = \{s_1 \dots s_k \mid k \in \mathbb{N} \wedge s_i \in L\}$$

Kleene-closure can also be defined as a fixed point!

$$\mathcal{L}(e)^* = L, \text{ where } f_e(L) = L$$

$$f_e(L) = L \circ \mathcal{L}(e) \cup \{\epsilon\}$$

$$(L^*)^* = L^*$$

This means kleene star is **idempotent**

$$\mathcal{L}(e)^* = L, \text{ where } f_e(L) = L$$

$$f_e(L) = L \circ \mathcal{L}(e) \cup \{\epsilon\}$$

Try an example:

L is the language of odd-length strings of zeros. Give a regex for L.

$$L = \{s_1 \dots s_{2k+1} \mid k \in \mathbb{N} \wedge s_i = 0\}$$

Try an example:

L is the language of odd-length strings of zeros. Give a regex for L.

$0(00)^*$

Try an example:

L is the language of all strings over alphabet $\{0,1\}$ where every 1 has an adjacent 1.

Try an example:

L is the language of all strings over alphabet $\{0,1\}$ where every 1 has an adjacent 1.

$$(0 \mid 111^*)^* \\ = (0 \mid 11^+)^*$$

e^+ is syntactic
sugar for ee^*

Try an example:

L is the language of odd decimal integers greater than zero. Give a regex for L.

Try an example:

L is the language of odd decimal integers greater than zero. Give a regex for L.

`(1|2|3|4|5|6|7|8|9)(0|1|2|3|4|5|6|7|8|9)*(1|3|5|7|9)|1|3|5|7|9`

Try an example:

L is the language of odd decimal integers greater than zero. Give a regex for L.

$[1 - 9][0 - 9]^*(1 | 3 | 5 | 7 | 9) | 1 | 3 | 5 | 7 | 9$

$[a-g]$ is a *character class*, and is syntactic sugar for $(a|b|c|d|e|f|g)$

Try an example:

L is the language of odd decimal integers greater than zero. Give a regex for L.

$([1 - 9][0 - 9]^*)(1 | 3 | 5 | 7 | 9)$

$e?$ is syntactic
sugar for $(e | \epsilon)$

Regexes in Python

```
import re
```

`r'a'` *a*

`r'ab'` *ab*

`r'b'` *b*

`r'a|b'` *a|b*

Regexes in Python

```
import re
```

<code>r' a* '</code>	a^*
----------------------	-------

<code>r' b+ '</code>	b^+
----------------------	-------

<code>r' c? '</code>	$c?$
----------------------	------

Regexes in Python

```
import re
```

`r'\d'` (0|1|2|3|4|5|6|7|8|9)

`r'[0-9]'` (0|1|2|3|4|5|6|7|8|9)

`r'z{2,4}'` (zz|zzz|zzzz)

Regexes in Python

```
import re
```

```
>>> m = re.match(r'\d', '5')
```

```
>>> m.group(0)
```

```
'5'
```


Regexes in Python

```
import re
```

```
>>> m = re.match(r'\d', '')
```

```
>>> m == None
```

```
True
```

Regexes in Python

```
import re
```

```
>>> m = re.match(r'(\d)\d\d', '456')
```

```
>>> m.group(0)
```

```
'456'
```

```
>>> m.group(1)
```

```
'4'
```

Regexes in Python

```
import re
```

```
>>> m = re.match(r'(\d)\d\d', '4567')
```

```
>>> m.group(0)
```

```
'456'
```

```
>>> m.group(1)
```

```
'4'
```

Regexes in Python

```
import re
```

```
>>> m = re.match(r'(\d)\d\d', '4567')
```

```
>>> m.group(0)
```

```
'456'
```

```
>>> m.group(1)
```

```
'4'
```

```
>>> m = re.match(r'^(\d)\d\d$', '4567')
```

```
>>> m == None
```

```
True
```